

Unit 3: Control Structure & Looping

Mastering Decision Making, Conditional Operators, Entry & Exit Control Loops, and Jump Statements
in C

Overview of Control Structures

Control structures determine the order in which individual statements, instructions, or function calls are executed or evaluated in a program.

Decision Control (Selection)

- ✓ **Definition:** Executes code blocks selectively based on boolean conditions.
- ✓ **Key Constructs:** if, if-else, else-if ladder, switch.
- ✓ **Evaluation:** Condition evaluates to *True (non-zero)* or *False (zero)*.
- ✓ **Real-Life Analogy:** A traffic signal light turning Red (Stop) or Green (Go).

Iterative Control (Looping)

- ✓ **Definition:** Repeats a block of code continuously until a condition is met.
- ✓ **Key Constructs:** for, while, do-while.
- ✓ **Classification:** Categorized into **Entry Control** and **Exit Control**.
- ✓ **Real-Life Analogy:** Running laps around a track until completing 5 full rounds.



Fundamental Rule: By default, C programs execute in a *sequential flow* (top to bottom). Control structures alter this default sequence to handle complex logical decisions.

Decision Making: If & If-Else

- ✓ **Simple `if`:** Executes the inner block *only if* the specified condition evaluates to true.
- ✓ **`if-else` Statement:** Provides a bi-directional flow: executing block A if true, and block B if false.
- ✓ **Non-Zero Rule:** In C language, any non-zero numeric value is treated as **TRUE**, while 0 is **FALSE**.
- ✓ **Single Statement Scope:** Without curly braces {}, only the single immediate statement falls inside the if block.
- ✓ **Boolean Operators:** Uses relational (>, <, ==, !=) and logical operators (&&, ||).

Real-Life Example: ATM Cash Withdrawal

If account balance $\geq$ requested amount, dispense cash; else display "Insufficient Funds".

```
// C Example: Voting Eligibility Check
#include

int main() {
    int age = 20;

    if (age >= 18) {
        printf("Eligible to vote!\n");
    } else {
        printf("Not eligible.\n");
    }
    return 0;
}
```



Multi-Way Decisions: Else-If & Nested If

Else-If Ladder

- ✓ Used when testing **multiple sequential conditions** in a top-down chain.
- ✓ As soon as one condition evaluates to true, its block executes and the rest of the ladder is skipped.
- ✓ The optional default else handles the scenario where all previous conditions evaluate to false.

```
if (marks >= 90) grade = 'A';  
else if (marks >= 75) grade = 'B';  
else grade = 'F';
```

Real-Life Analogy: Academic Grading

Evaluating student percentage into Distinction, First Class, Pass, or Fail.

Nested If Statements

- ✓ An if statement placed inside another if or else block.
- ✓ Evaluates secondary conditions *only after* the primary condition passes successfully.
- ✓ Dangling-Else Problem: An else always pairs with the nearest un-paired if unless grouped by {}.

```
if (user_verified) {  
    if (pin_correct) {  
        grantAccess();  
    }  
}
```

Real-Life Analogy: Security Check

First verify ID card validity; if valid, check fingerprint access.

Switch Statement & Ternary Operator

Switch-Case Statement

- ✓ Multi-way selection based on matching an **integral/character expression** against constant values.
- ✓ **Constant Rule:** Case labels must be unique constant integers or char literals (no variables/ranges).
- ✓ **Break Keyword:** Prevents *fall-through* execution to subsequent cases.

```
switch(day) {  
    case 1: printf("Mon"); break;  
    default: printf("Other");  
}
```

🍴 Real-Life Example: Restaurant Menu

Pressing '1' selects Pizza, '2' selects Burger, '3' selects Pasta.

⚡ Ternary / Conditional Operator

- ✓ The **only ternary operator** in C (takes 3 operands): Condition ? Exp1 : Exp2.
- ✓ Shorthand replacement for simple if-else assignment statements.
- ✓ Returns Exp1 if condition is true, otherwise returns Exp2.

```
// Syntax: (condition) ? value_if_true : value_if_false  
int max = (a > b) ? a : b;
```

💡 Real-Life Example: Pass/Fail Status

```
status = (marks >= 40) ? "PASS" : "FAIL";
```

LOOPING CONSTRUCTS

Iteration Control Structures

Executing code blocks repeatedly with precision, efficiency, and automated termination conditions.

1. Initialization

Sets the loop counter variable to its starting initial value.

2. Test Condition

Evaluates if iteration should continue or exit immediately.

3. Updation

Increments or decrements the counter variable per pass.

Entry Control Loops: While & For

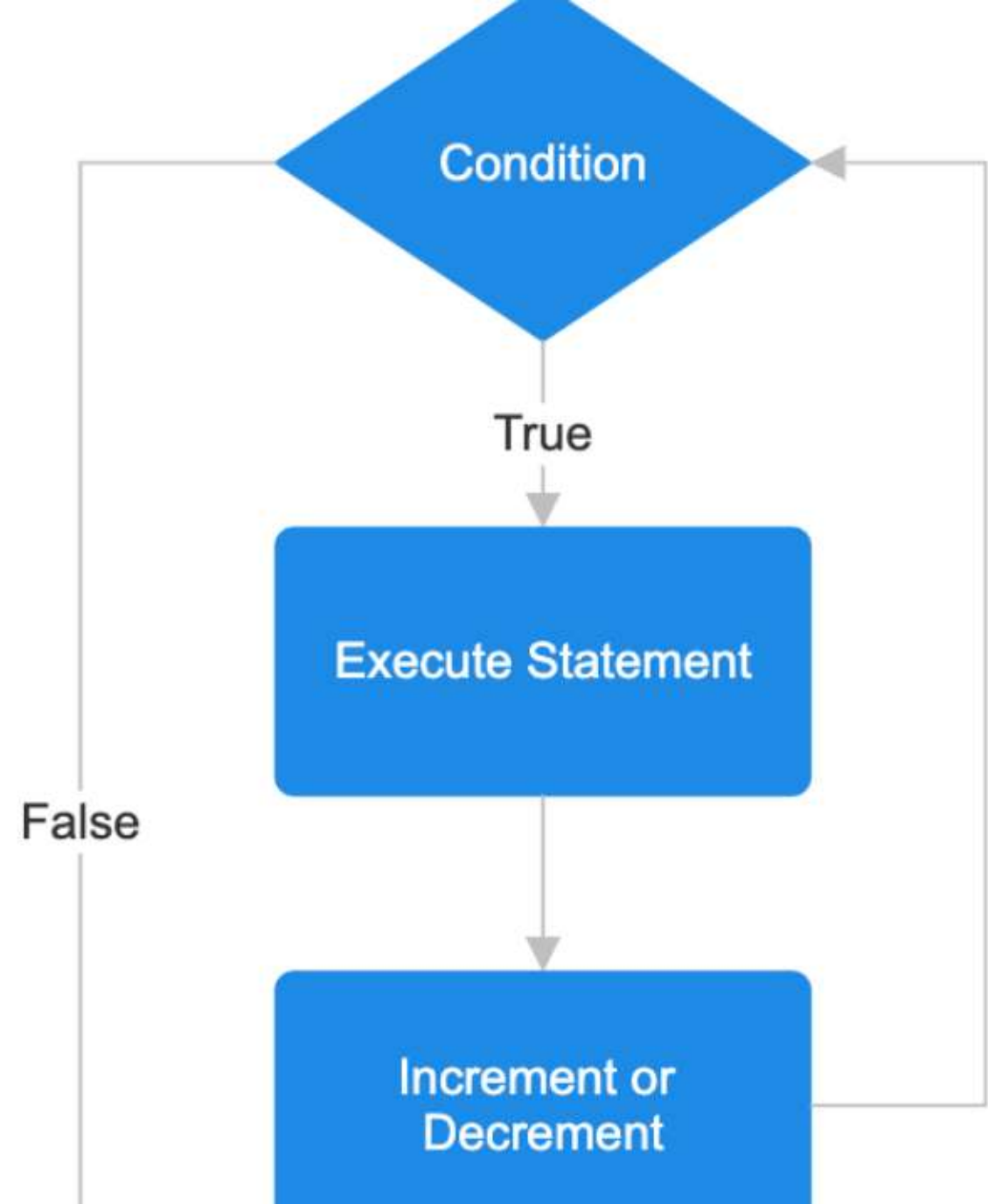
Entry control loops test the condition **before** executing the loop body. If initial test fails, body executes 0 times.

```
// For Loop: Fixed Iterations
for (int i = 1; i <= 5; i++) {
    printf("Count: %d\n", i);
}

// While Loop: Dynamic Iterations
while (attempts < 3) { login(); attempts++; }
```

🔑 Real-Life Analogy: Password Login Retry

Checking remaining security login attempts before allowing password entry screen.



Exit Control Loop: Do-While

- ✓ **Guaranteed Execution:** Tests condition *after* executing the body. Runs **at least once**.
- ✓ **Syntax Requirement:** Mandatory semicolon ; required after closing while(condition); statement.
- ✓ **Post-Test Nature:** Ideal when the first execution must happen regardless of condition state.
- ✓ **Use Case:** Menu-driven applications where options must be shown before user input.

Real-Life Example: Game "Play Again" Prompt

Playing at least 1 full round before asking player: "Do you want to retry? (Y/N)".

```
// C Program: Interactive Menu using do-while
#include

int main() {
    int choice;
    do {
        printf("\n--- MAIN MENU ---\n");
        printf("1. Start Game\n2. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
    } while (choice != 2);

    return 0;
}
```




Entry Control vs Exit Control Loops

Comparison Parameter	Entry Control Loops (while / for)	Exit Control Loop (do-while)
Condition Checking	Tested before loop body execution.	Tested after loop body execution.
Minimum Executions	0 times (if condition is false initially).	At least 1 time (always executed).
Semicolon Syntax	No semicolon after condition header.	Mandatory semicolon after while(cond);
Control Variable	Must be initialized <i>before</i> condition test.	Can be initialized or modified inside the body.
Primary Best Use Case	Pre-calculated limits, array processing.	Menu selection prompts, input validation.
Examples	for (i=0; i / while(cond)	do { ... } while(cond);

↓ Concept of Jump: Break, Continue & Goto



Break Statement

Immediately **terminates** the innermost loop or switch block. Control passes to the next statement outside.

```
for(int i=1; i<=10; i++) {  
    if(i == 5) break;  
    printf("%d ", i);  
}  
// Output: 1 2 3 4
```



Continue Statement

Skips remaining statements in current iteration and jumps directly to the next iteration loop check.

```
for(int i=1; i<=5; i++) {  
    if(i == 3) continue;  
    printf("%d ", i);  
}  
// Output: 1 2 4 5
```



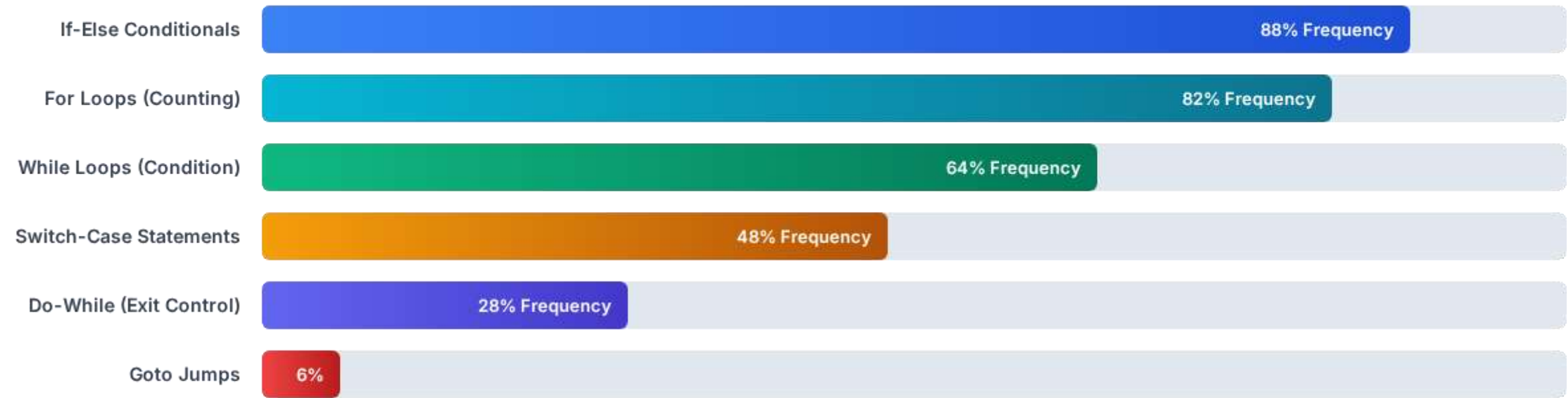
Goto Statement

Unconditional jump transfer to a defined label:. Discouraged in modern engineering (creates spaghetti code).

```
if (error) goto cleanup;  
  
cleanup:  
    closeFiles();
```

Control Construct Usage in Algorithms

Estimated relative adoption frequency of control structures across typical B.Tech introductory C algorithms:



Note: for loops and if-else blocks make up over 70% of control operations in competitive programming and fundamental software design.

Unit 3 Key Takeaways & Discussion

Exam Questions:

1. Differentiate between **Entry Control** and **Exit Control** loops with suitable code
2. Explain the dangling else problem and how curly braces resolve it.
3. What is the difference between break and continue statements?
4. Rewrite an if-else grade checking program using the ternary operator ?.